

PRACTICAL-03

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

CODE:-

```
venu@LAPTOP-KHLR30TF: ~/os-lab/practical$ cat practical-03.c

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("=== PROCESS CREATION USING fork() ===\n");

    printf("\nBefore fork():\n");
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        printf("State: Running\n");

        printf("Child State: Waiting/Sleeping\n");
        sleep(10);

        printf("Child State: Running again\n");
        sleep(5);

        printf("Child State: Terminated\n");
        exit(0);
    }

    if (pid == 0)
    {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        printf("State: Running\n");

        printf("Child State: Waiting/Sleeping\n");
        sleep(10);

        printf("Child State: Running again\n");
        sleep(5);

        printf("Child State: Terminated\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("\n--- PARENT PROCESS ---\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        printf("Child PID : %d\n", pid);
        printf("State : Running\n");

        printf("Parent State: Waiting for child...\n");

        wait(NULL);

        printf("Child process has terminated.\n");
        printf("Parent State: Running\n");

        printf("Parent State: Terminated\n");
    }

    return 0;
}

venu@LAPTOP-KHLR30TF:~/os-lab/practical$
```

OUTPUT:-

```
venu@LAPTOP-KHLR30TF: ~/  X  +  v
venu@LAPTOP-KHLR30TF:~$ cd os-lab
venu@LAPTOP-KHLR30TF:~/os-lab$ cd parctical
-bash: cd: parctical: No such file or directory
venu@LAPTOP-KHLR30TF:~/os-lab$ cd practical
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ nano practical-03.c
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ gcc program-03.c -o program-03
cc1: fatal error: program-03.c: No such file or directory
compilation terminated.
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ gcc practical-03.c -o practical-03
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ ls
destination.txt practical-03 practical-03.c program program-01.c program-02 program-02.c program.c source.txt
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ ./practical-03
=== PROCESS CREATION USING fork() ===

Before fork():
PID : 779
PPID : 447

--- PARENT PROCESS ---
PID : 779
PPID : 447
Child PID : 780
State : Running
Parent State: Waiting for child...

--- CHILD PROCESS ---
PID : 780
PPID : 779
State: Running
Child State: Waiting/Sleeping

Child State: Running again

Child State: Terminated
Child process has terminated.
Parent State: Running
Parent State: Terminated
venu@LAPTOP-KHLR30TF:~/os-lab/practical$
venu@LAPTOP-KHLR30TF:~/os-lab/practical$
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ cat practical-03
ELF>'@?7@8@@@tttmm  *x-x=x=***-*=*ppp$*$"*@*"*" Setd*"*"@P*td*!@!@!44Q*tdR*tdx-x=x=**GNU"*?7?bA***xka*jf*/lib64/ld-linux-x86-64.so.2**e*mlz *R
```